

Creazione di Funzioni e Trigger in pgAdmin con PostgreSQL

Basi di Dati I

PostgreSQL permette di creare sia funzioni che trigger utilizzando l'interfaccia grafica di pgAdmin, fornendo un modo user-friendly per lavorare con il codice PL/pgSQL senza dover scrivere statement SQL grezzi.

- Creazione di funzioni tramite GUI
- Creazione di trigger
- Definire il codice PL/pgSQL nell'interfaccia

Creazione di Funzioni in pgAdmin

Utilizzo dell'Interfaccia GUI

Per creare una funzione usando l'interfaccia di pgAdmin:

- 1 **Navigare al dialogo di creazione funzione:** Servers → Database → Schema → Functions → click destro → Create → Function
- 2 **Configurare le proprietà di base**
- 3 **Impostare la definizione della funzione**
- 4 **Aggiungere parametri**
- 5 **Scrivere il codice della funzione**

Configurazione Proprietà di Base

Proprietà da configurare:

- **Name**: Specificare il nome della funzione (es. "inc_by_n")
- **Owner**: Impostare il proprietario della funzione
- **Schema**: Scegliere lo schema appropriato
- **Comment**: Aggiungere una descrizione opzionale

Definizione della Funzione

Impostazioni della definizione:

- **Return type**: Selezionare il tipo di dato che la funzione restituirà (es. `integer`)
- **Language**: Scegliere “`plpgsql`” per le funzioni PL/pgSQL
- **Parameters**: Aggiungere parametri di input con i loro tipi e modalità

Come aggiungere parametri:

- Cliccare “Add” per creare parametri
- Specificare nome parametro, tipo di dato e modalità (IN, OUT, INOUT)
- Impostare valori di default se necessario

Scrittura del codice:

- Nel tab “Code”, scrivere il codice PL/pgSQL (logica della funzione)
- Usare statement BEGIN ed END
- Esempio: `BEGIN RETURN num + val; END;`

Esempio di Creazione Funzione

- L'interfaccia pgAdmin genera questo SQL equivalente:

```
CREATE OR REPLACE FUNCTION inc_by_n(  
    num integer,  
    val integer DEFAULT 1  
)  
RETURNS integer  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RETURN num + val;  
END;  
$$;
```

Creazione di Trigger in pgAdmin

- La creazione di trigger coinvolge due passaggi:
 - 1 **Creare una funzione trigger:** Prima, creare una funzione che sarà eseguita dal trigger
 - 2 **Creare il trigger:** Collegare la funzione a una tabella e specificare quando deve attivarsi

- Mentre pgAdmin non ha una GUI dedicata per la creazione di trigger come per le funzioni, è possibile usare il Query Tool:
 - 1 Click destro sul database e selezionare "Query Tool"
 - 2 Scrivere il codice del trigger nel Query Editor
 - 3 Eseguire usando il pulsante Execute/Refresh (F5)

Esempio Creazione Trigger – Tabelle

■ Creazione delle tabelle:

```
-- Creare le tabelle
CREATE TABLE record(
    ID INT PRIMARY KEY NOT NULL,
    NAME TEXT NOT NULL,
    AGE INT NOT NULL,
    ADDRESS CHAR(50),
    SALARY REAL
);

CREATE TABLE record_book(
    EMP_ID INT NOT NULL,
    ENTRY_DATE TEXT NOT NULL
);
```

Esempio Creazione Trigger – Funzione

■ Creazione della funzione trigger:

```
-- Creare la funzione trigger
CREATE OR REPLACE FUNCTION record_book_func()
RETURNS TRIGGER AS $example_table$
BEGIN
    INSERT INTO record_book(EMP_ID, ENTRY_DATE)
    VALUES (new.ID, current_timestamp);
    RETURN NEW;
END;
$example_table$ LANGUAGE plpgsql;
```

Esempio Creazione Trigger - Trigger

■ Creazione del trigger:

```
-- Creare il trigger
CREATE TRIGGER postgres_trigger
AFTER INSERT ON record
FOR EACH ROW
EXECUTE PROCEDURE record_book_func();
```

- pgAdmin fornisce diverse funzionalità UI per lavorare con codice PL/pgSQL:
 - **Query Editor**: Scrivere e modificare codice PL/pgSQL con evidenziazione sintassi
 - **Pulsante Execute/Refresh**: Eseguire il codice direttamente dall'interfaccia
 - **Pannelli risultati**: Visualizzare risultati di esecuzione e messaggi di errore
 - **Visualizzatore piano esecuzione**: Analizzare performance delle query

- **Gestione tramite interfaccia:**
- Una volta create, le funzioni appaiono nella struttura ad albero di pgAdmin:
 - Navigare a Servers → Database → Schema → Functions
 - Click destro e selezionare "Refresh" se le funzioni appena create non appaiono
 - Visualizzare proprietà e codice delle funzioni tramite la GUI

Cos'è una Funzione Trigger?

- Una funzione trigger è una **funzione speciale** definita dall'utente
- Viene eseguita **automaticamente** quando si verifica un evento specifico
- Deve essere dichiarata con tipo di ritorno **TRIGGER**
- Non accetta argomenti nella dichiarazione

Sintassi Base di una Funzione Trigger

```
CREATE OR REPLACE FUNCTION nome_funzione()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Logica del trigger  
    RETURN NEW; -- o OLD o NULL  
END;  
$$;
```

Caratteristiche delle Funzioni Trigger

- Nessun argomento nella dichiarazione della funzione
- Tipo di ritorno obbligatorio: trigger
- Accesso a variabili speciali come NEW, OLD, TG_OP
- Possono essere riutilizzate per piú trigger

Esempio: audit function

```
CREATE OR REPLACE FUNCTION auditfunc()  
RETURNS TRIGGER AS $my_table$  
BEGIN  
    INSERT INTO Price_Audits(book_id, entry_date)  
    VALUES (new.ID, current_timestamp);  
    RETURN NEW;  
END;  
$my_table$ LANGUAGE plpgsql;
```

- Questa funzione inserisce un record di audit ogni volta che viene attivata

Valori di ritorno delle funzioni trigger

- **BEFORE triggers**: Il valore di ritorno determina se l'operazione continua
 - **NEW**: Continua con la riga modificata
 - **NULL**: Annulla l'operazione
- **AFTER triggers**: Il valore di ritorno viene ignorato
- **INSTEAD OF triggers**: Solo per le viste

Esempio: validazione dati

```
CREATE FUNCTION emp_stamp() RETURNS trigger AS $emp_stamp$
BEGIN
    -- Verifica che empname non sia NULL
    IF NEW.empname IS NULL THEN
        RAISE EXCEPTION 'empname cannot be null';
    END IF;

    -- Aggiorna timestamp e utente
    NEW.last_date := current_timestamp;
    NEW.last_user := current_user;
    RETURN NEW;
END;
$emp_stamp$ LANGUAGE plpgsql;
```

Creazione del Trigger

```
CREATE TRIGGER emp_stamp  
  BEFORE INSERT OR UPDATE ON emp  
  FOR EACH ROW  
  EXECUTE FUNCTION emp_stamp();
```

- Il trigger `emp_stamp` usa la funzione `emp_stamp()`
- Si attiva BEFORE INSERT o UPDATE
- Esegue la funzione per ogni riga modificata

Variabili Speciali nei Trigger

- **NEW**: Nuovi dati della riga (INSERT/UPDATE)
- **OLD**: Dati precedenti della riga (UPDATE/DELETE)
- **TG_OP**: Tipo di operazione (INSERT, UPDATE, DELETE)
- **TG_WHEN**: Timing del trigger (BEFORE/AFTER)
- **TG_TABLE_NAME**: Nome della tabella interessata